Hyderabad Property Price Prediction

Data Cleaning: Problems Found, Decisions Taken, Results Achieved

Phase 1 of 4 | Source: MagicBricks listings, scraped into SQLite

1. Context

The objective is a model that predicts the **asking price** of residential property in Hyderabad. The data was scraped from MagicBricks in price bands (1-50 lakh, 50-80 lakh, 80 lakh-1 crore, and upward) and stored in a SQLite database, `listings_master.db`.

Two tables matter. `listings` holds 20,700 property records across 57 columns; `locality_ratings` holds 2,009 unique localities across 38 columns of amenity ratings and area-level price statistics. They were joined with a left join on `locality_id`, preserving every listing whether or not its locality had a ratings record, producing a working table of 20,700 rows and 94 columns.

This document covers the cleaning phase only. Exploratory analysis, feature engineering and modelling follow separately.

2. Headline Result

Measure	Before	After	Change
Rows	20,700	19,715	95.2% retained
Columns	94	71	23 removed
Max price	Rs 2,50,000 crore	Rs 49 crore	data-entry errors excluded
Std. dev / median price	149	1.41	distribution now realistic
Mean latitude	10.9 (Indian Ocean)	17.44 (Hyderabad)	placeholders removed
Areas below 200 sq ft	239	0	unit bug corrected, typos cut
Stored size	36.5 MB (CSV)	6.5 MB (Parquet)	dtypes preserved on reload

Only 4.8% of rows were removed. Every removal is individually counted and justified in section 5.

3. Working Method

Three rules governed the whole phase, and they are the part worth defending in review:

- **Audit before acting.** Every transformation was preceded by a diagnostic that measured the problem, and followed by a check that confirmed the fix. No column was dropped on the strength of its name alone.
- **Count every row that leaves.** Each filter prints the count removed and the reason, so the final row count is fully reconstructable rather than merely asserted.
- **Separate corrections from decisions.** Fixing a wrong unit is a correction. Excluding properties above a price threshold is a modelling decision that narrows what the model is accountable for. The two are documented differently.

4. Problems Found and Resolved

4.1. Rows with no target variable

Problem. 280 listings had no price. A row with no target cannot be a training example.

How it surfaced. A null count on `price`, then a cross-tabulation against `price_label` to find out what kind of rows they were: 263 were labelled “Call for Price” (the seller deliberately withheld it) and 17 had no price information of any kind.

Decision. Drop all 280. A missing target cannot be imputed — imputing it means inventing the answer and then training a model to reproduce the invention. The question worth asking instead is whether the target is *recoverable*: 4 rows did carry both a price-per-square-foot and an area, making the price algebraically recoverable. Recovering them was rejected as not worth the unit-conversion risk for 0.02% of the data.

Action. Filtered on `price.notna()`, with the breakdown of what was leaving printed first.

Result. 20,700 -> 20,420 rows.

4.2. Columns carrying no information

Problem. 94 columns, many of them scrape artefacts, empty fields or duplicates of one another.

How it surfaced. A scan for columns with a single distinct value found four constants (`city`, `is_Luxury`, `has_rera`, `detail_scraped`). A null-rate ranking found three columns under 3% populated. An `.equals()` test showed `lat`/`long` to be byte-identical to `locality_lat`/`locality_long`.

Decision. Drop constants, near-empty columns, scrape metadata and exact duplicates — but **explicitly refuse a blanket missingness threshold**. `parking` (76% missing), `power_status` (74%) and `water_status` (73%) were kept, because a missing parking field most likely means no parking. Missingness is itself a signal; a “drop anything over 50% null” rule would have discarded it.

Action. Dropped 20 columns in four named groups, each group commented with its justification. One near-miss: `pmt_source` looked like an opaque numeric code, but its values turned out to be `android`, `Web`, `Telecalling` — the posting channel, a plausible signal about seller type. It was kept. Columns are judged on their values, not their names.

Result. 94 -> 74 columns, with the informative sparse columns intact.

4.3. Area was not in the unit its column name claimed

Problem. `covered_area_sqft` did not hold square feet. It held the area in whatever unit the seller selected, with a companion column, `area_type`, naming that unit — sometimes as a label (`Sq-ft`, `Sq-yrd`), sometimes as an unresolved numeric code (`12800`, `12803`). Roughly 10% of rows therefore had an area understated ninefold. Left alone, this is the single most damaging error in the dataset: area is the strongest numeric predictor of price.

How it surfaced. The site publishes its own `price_per_sqft`, which gives a way to reverse-engineer the unit without any external reference. For each row, `(price / area) / price_per_sqft` yields the factor the site itself applied. Grouped by `area_type`, the median ratio identified every code: 12800 = 1.0, 12803 = 9.0, 12801 = 10.764, 12817 = 435.6 (cents).

Decision. The same ratio computed *per row* exposed a second and larger problem: on **463 rows the label itself was wrong** — 218 labelled `Sq-yrd` actually held square feet, and 237 labelled `Sq-ft` held square yards. Corroborated independently: one row labelled `Sq-ft` had a covered area of 109 against a carpet area of 900, which is physically impossible. The decision was to **trust the arithmetic over the metadata**, using the label only where the ratio was unavailable.

Action. A per-row factor was derived by snapping the measured ratio to the nearest known unit within a 5% tolerance, falling back to the label map. One deliberate override: for square metres the site's own figure uses a factor of 9, which is wrong, so the physical constant 10.7639 was applied instead — the only place where a known external truth was allowed to outrank the data. Separately, 106 rows were rescued by stripping thousands separators before numeric conversion; without it they would have silently become nulls.

Result. Implausible areas fell sharply: under 200 sq ft from 239 to 137, over 20,000 sq ft from 128 to 21. The decisive check was that group medians converged — code 12803 moved from 178 to 1,602 sq ft, landing alongside the square-foot groups, because a 3-BHK is the same size whichever unit the seller typed.

4.4. Numeric attributes stored as text

Problem. `bedrooms`, `bathrooms`, `balconies` and `floor` were strings, so no model could use them and no summary statistic was meaningful.

How it surfaced. Value counts on each column revealed two distinct obstacles: a censored bucket `'> 10'` (277 bedrooms, 234 bathrooms, 43 balconies) and three text floor labels (`Ground` 1,712, `Upper Basement` 65, `Lower Basement` 64).

Decision. The `'> 10'` bucket is right-censored data — the true value could be 11 or 25. Mapping it to 11 preserves the ordering and keeps the row; mapping it to null would discard the very fact that the property is large. Floor labels were mapped to 0, -1 and -2 so that the ordinality survives, a basement genuinely sitting below ground level.

Action. Converted all four columns to numeric. `bhk` was then dropped, but only after a grouped check proved it a one-to-one re-encoding of `bedrooms` (11700->1 through 11710->'>10'), so the deletion rests on an assertion in the notebook rather than on an assumption.

Result. All five count columns numeric. Validation: `floor` exceeded `total_floors` on only **3 of 14,253** rows — a broken parse would have produced thousands, so the low count is positive evidence the conversion was correct.

4.5. Coordinates: a null disguised as a valid value

Problem. 7,531 rows (37%) carried latitude and longitude of exactly `0, 0`. This is more dangerous than a missing value, because it is numerically valid and silently corrupts every statistic computed over it. The symptom was visible in the initial profiling: mean latitude 10.9, a point in the Indian Ocean.

How it surfaced. A bounding-box test against Hyderabad's real extent (16.8-18.0 N, 77.8-79.0 E) separated valid points from the rest, and grouping the failures by coordinate showed they were not random noise but repeated identical values from a failing geocoder.

Decision. Inspect before deleting. The 113 non-zero out-of-box rows split into recognisable groups: 4 had **latitude and longitude transposed** and were recoverable; the remainder were genuine geocoding failures, including points in London (51.51, 0.12), Delhi NCR (28.53, 77.40) and a ~30-row cluster in Warangal reused across unrelated Hyderabad localities. Recover the 4, null the rest.

Action. The transposition test is generic — a row is swapped when its latitude falls in the longitude range and vice versa — rather than hardcoded to specific rows. Invalid points on **both** coordinate pairs were nulled (the locality centroids had 105 bad values of their own), then listings without a usable point inherited their locality centroid, with a boolean flag recording the substitution so the model can distinguish exact from approximate.

Result. 12,780 rows with their own coordinates, 6,110 on a locality fallback, 1,530 left null. No rows were dropped — a listing without a coordinate still has a locality, a size and a price. Mean latitude moved from 10.9 to 17.44, standard deviation 0.08.

4.6. Duplicate listings invisible to the obvious check

Problem. The same property appears multiple times, but a conventional duplicate check finds nothing.

How it surfaced. A duplicate check across all columns except `property_id` returned zero — a false negative, because `detail_url`, `raw_json` and `scraped_at` are unique per scrape by construction and guarantee every row looks distinct. Re-running on a *content* key (locality, type, bedrooms, area, price, floor, project) found 517 rows in 236 groups.

Decision. Whether to include price in the key was the real decision: with it, 281 rows are removable; without it, 952. The conservative key was chosen, because excluding price assumes no two distinct units in a 500-flat township ever share a locality, size, floor and project — a poor assumption in Hyderabad's apartment market. An honest ambiguity remains: `owner_name` differs in 220 of the 236 groups, so these may be one flat listed by two brokers, or two identical flats. Nothing in the data distinguishes them.

Action. The justification does not depend on resolving that ambiguity. Identical feature vectors with identical targets split across training and test inflate the test score regardless of which situation produced them. Rows were sorted by `listing_date` descending before deduplication, so the surviving copy carries the seller's most recent asking price.

Result. 20,420 -> 20,139 rows, with a re-run of the same check confirming zero remaining.

4.7. Target leakage, including a form hidden in free text

Problem. Features that encode the target make a model look excellent in validation and fail in production.

How it surfaced. `price_per_sqft` was tested directly against `price / covered_area`: correlation 1.0, with only 6 rows differing by more than 1% — 5 of those being the deliberate square-metre override from 4.3. It is the target divided by a feature.

Decision. Three severities were separated rather than treated alike. **(a) Exact leakage**: `price_per_sqft` and `price_label` were dropped. **(b) Site-published locality aggregates** (average price per sq ft, quarterly price movement) were *kept*: these are published before any individual listing exists, so they genuinely are available at prediction time, which is the real test for leakage. The caveat — that they were computed over a pool overlapping our rows, making cross-validation mildly optimistic — is documented in the notebook rather than left implicit. **(c)** Any target encoding computed from our own data, such as mean price per locality, is genuine leakage and must be fitted on training folds only.

Action. A fourth form was then tested for and found. MagicBricks auto-generates the `description` text, and a check of 3,000 rows showed **2,592 of them (86%) state the price verbatim** — for example: “this dec '28 apartment is sold at price of Rs 46.9 Lac”.

Result. Two columns dropped, a documented keep-list for the aggregates, and a hard constraint recorded for the next phase: any text feature built on `description` or `property_title` must have the price stripped first, or it will carry the answer.

4.8. Implausible prices, and why percentiles were the wrong tool

Problem. Maximum price of Rs 2.5 lakh crore against a median of Rs 1.5 crore; areas ranging from 1 to 560,000 sq ft.

How it surfaced. Checking `price` against the site's own `price_label` showed 99.99% agreement — the extreme row was literally labelled “25000 Cr” on MagicBricks. These are **seller data-entry errors, not scraping faults**, which rules out recovery and leaves exclusion as the only option.

Decision. Filter on implied price per square foot rather than on price, since a Rs 40 crore listing is ordinary at 18,000 sq ft and absurd at 1,200. The instinctive approach — clipping the top and bottom 1% — was rejected on inspection: the 99th percentile of implied price per sq ft sits at Rs 75,969, while Hyderabad's most expensive real estate trades near Rs 25,000-30,000. **The tail is already contaminated, so the percentile is itself a corrupted statistic** and quantile clipping would bake the errors in. Domain knowledge set the bounds instead.

Action. Each candidate bound was priced before being chosen (Rs 1,500-40,000 costs 384 rows; Rs 2,000-30,000 costs 521). The loosest defensible bound was taken, on the principle that a surviving outlier costs a little accuracy whereas deleting a legitimate luxury segment makes the model unable to price it at all. 1,509 rows have no area and so cannot be ratio-checked; they were kept under a flat Rs 50 crore ceiling, which removed 3 rows.

Result. 20,139 -> 19,715 rows. The ratio of standard deviation to median price fell from 149 to **1.41** — the signature of a distribution no longer governed by a single bad row.

5. Row Ledger

Every row that left the dataset, and why. The totals reconcile to the final count.

Step	Reason	Rows removed	Remaining
Start	Merged listings and locality ratings	—	20,700
4.1	No price (263 “Call for Price”, 17 blank)	280	20,420
4.6	Duplicate / relisted properties	281	20,139
4.8	Area outside 200-20,000 sq ft	152	19,987
4.8	Implied price/sq ft outside Rs 1,500-40,000	269	19,718
4.8	No area and price above Rs 50 crore	3	**19,715**

> Note that the price-per-sq-ft filter removed 269 rows rather than the 384 forecast in isolation, because the area filter had already caught 115 of them. The ratio was recomputed between the two filters rather than reused.

6. Deliverable and Reproducibility

The cleaned dataset is written to `data/processed/` in two formats: Parquet at 6.5 MB, which preserves dtypes and loads quickly, and CSV at 36.5 MB for inspection. The distinction matters: CSV is a text format with no concept of a dtype, so a parsed datetime column returns as a string on reload and the work of converting it is silently undone.

A read-back check confirms the saved file matches the in-memory frame on shape and dtypes. The acceptance test for the notebook as a whole is a restart-and-run-all from an empty kernel: until it reproduces 19,715 rows from scratch, the pipeline is a product of one particular execution order rather than of the code.

7. Limitations Carried Forward

Stated explicitly so that the next phase inherits them as constraints rather than discovering them as surprises.

- **Sampling bias by design.** The scrape was organised by price band, so the marginal distribution of price reflects how many pages each band offered, not the Hyderabad market. Valid for a conditional model of price given features; not a basis for claims about the market, and a reason for care if stratifying a split on price.
- **Coordinates are locality-resolution at best.** A single coordinate pair covers 277 listings, 253 of them in one locality, so many values in the listing columns are already centroids. Geospatial features should not be expected to add much beyond a well-built locality feature.
- **Two locality columns, different granularity.** `locality` is the listing's free text (2,427 distinct, “Patancheruvu”); `locality_name` is canonical from the ratings table (1,911 distinct, “Patancheru”) but null wherever the join failed. The canonical column is the better grouping key, with a fallback.
- **1,509 rows retained without an area**, excluded from the plausibility check. The lowest surviving price, Rs 1.45 lakh, is among them and should be revisited once the target distribution is plotted.
- **Validation will read slightly optimistic** wherever the site-published locality aggregates are used, most of all in localities with few listings behind the average.

8. Interview Summary

The four findings that best demonstrate judgement rather than procedure.

Metadata is a claim, not a fact

The area column was not in the unit its name promised, and the companion column naming the unit was itself wrong on 463 rows. Rather than trusting either, the site's own published price-per-square-foot was used to reverse-engineer the true factor per row, with a physical constant overriding the data in the one case where the site's arithmetic was demonstrably wrong. **The useful instinct: when two sources disagree, find a third that is derived differently.**

Leakage can hide in prose

Dropping `price_per_sqft` is the obvious half. The non-obvious half was testing the auto-generated description text and finding that 86% of listings state the price in words. A text feature built without checking would have scored superbly and been worthless. **Leakage is a property of the information, not of the data type.**

Know when the data cannot answer the question

The reflex for outliers is to clip at the 1st and 99th percentiles. Here the 99th percentile of implied price per sq ft was Rs 75,969 against a real-world ceiling near Rs 30,000 — the tail was already contaminated, so the quantile could not be trusted to locate it. Domain knowledge had to set the bound. **A statistic computed over corrupted data inherits the corruption.**

Not all leakage is equal

The locality price aggregates were kept, not dropped. They are published before any individual listing exists, so they pass the real test — would this value be available at prediction time? The mild contamination was documented instead of being used as grounds to discard a genuinely useful feature. **The opposite error, over-pruning out of caution, costs accuracy just as surely.**

On process

Three bugs during the work were instructive in their own right: a `drop(inplace=True)` that made the notebook fail on re-run, a `df.drop(...)` missing its reassignment so that a cell appeared to succeed while changing nothing, and a repeated cell raising `KeyError` on columns already removed. All three share a cause: a notebook's state depends on execution order, and the only defence is making each cell re-runnable and validating with a restart-and-run-all.

Phase 1 of 4. Next: exploratory analysis on the cleaned dataset, then feature engineering, then modelling.